Target Multi-Account (Tenant) Data Model — To-Be Design

Revision: 1 **Last modified:** 2026-07-10T11:18:54Z

> **Scope.** This is the KEYSTONE data-model design for adding an ACCOUNT (tenant) layer to the Helix OTA control plane (`server/`, Go/Gin modular monolith). It designs the to-be entities (Account, User, Membership, Role/Permission), how they scope the existing OTA model (Device/Artifact/Release/Deployment/Telemetry/Group), the `store.Repository` extension for BOTH the in-memory MVP and the pgx target, a migration-style DDL sketch, the OQ-4 migration path, and the super-admin at the data layer. It is the single source of truth the sibling to-be docs (`21_authz_rbac_superadmin`, `22_api_surface`, `24_project_side_cli`, `26_security_threat_model`, `30_delivery_plan`) reference — so it is deliberately precise and self-consistent (§11.4.186 anti-divergence).
>
> **Reading order.** Read `00_INDEX.md` (mandate + open questions) and `10_existing_auth_and_project_model.md` (the authoritative as-is facts) FIRST. Every as-is claim below is grounded in a `file:line` those docs already established; this doc does not re-derive them, it cites them. Every design choice is grounded in a cited source (`## Sources verified 2026-07-10`) or marked "original work". Every open choice is presented as a recommendation-with-tradeoffs for operator decision — never a silent decision (§11.4.6).

---

0. The one decision this doc anchors: tenancy model

Three canonical multi-tenant storage models exist (Microsoft Azure Architecture Center; AWS; dasroot 2026): **(1) shared database + shared schema** with a `tenant_id`/`account_id` column on every tenant-owned row and database-enforced row scoping; **(2) schema-per-tenant**; **(3) database-per-tenant**.

Model	Isolation	Ops cost	Cross-tenant analytics	Fit for Helix OTA today
Shared schema + `account_id` (+ RLS)	Policy-enforced; a policy/where bug is a leak	Lowest — one schema, one migration	Trivial (one table)	**Best** — matches ADR-0003 "modular monolith — one PostgreSQL schema" (`001_initial_schema.up.sql:16`) and the existing single-schema store (`schema_postgres.sql:11`)
Schema-per-tenant	Strong; per-tenant customization	Every migration ×N tenants	Harder	Overkill; no per-tenant schema customization is required by the mandate
Database-per-tenant	Perfect; per-tenant residency	N connection pools, N backups	Hardest	Reserve for a future compliance/residency tenant only

Recommendation (KEYSTONE): shared schema + a denormalized `account_id` column on every tenant-owned table, defended by PostgreSQL Row-Level Security (RLS) at the pgx layer. This is the industry default when tenant counts can grow and no per-tenant schema customization is required (Microsoft, AWS, dasroot all recommend shared-schema+RLS as the default; escalate only for compliance/residency). It also requires the *least* change to the shipped single-schema store.

Tradeoff to accept: isolation now depends on correct scoping + correct RLS policies; a forgotten `WHERE account_id = …` in application code would leak cross-tenant — which is exactly why RLS (database-enforced, "works even when your application code has bugs" — AWS) is the belt-and-suspenders second layer, and why `26_security_threat_model.md` must carry a paired mutation proving a non-scoped session cannot read another account's rows.

Escalation path (operator decision, not now): a single high-compliance tenant that needs data residency or a separate backup schedule can later be promoted to its own schema or database without changing the shared-schema code for everyone else — the `account_id` seam is forward-compatible with a routing layer that maps some accounts to isolated storage.

---

1. Entities

Four new/lifted entities. Id type is **opaque `TEXT`** to match the *shipped* store convention (`project_id`, `device_id`, `artifact_id` are all `TEXT` — `schema_postgres.sql:15,34,49,240`), NOT the `UUID` used by the canonical design-target migration (`001_initial_schema.up.sql:40` etc.).

> **Divergence to reconcile (§11.4.186).** The shipped store (`schema_postgres.sql`) uses opaque `TEXT` ids; the canonical target migration (`001_initial_schema.up.sql`) uses `UUID` and already declares `users` + `api_keys` tables (10_* §3, `001_initial_schema.up.sql:39-71`) that were never carried into the shipped store. **Recommendation:** the *implementation* seam (this design, §3–§4) uses opaque `TEXT` to mirror the store it actually extends; the canonical `001`-style schema keeps `UUID`. Both remain internally consistent; §4 states the mapping so the two representations never silently diverge. Tradeoff: `TEXT` ids forgo `gen_random_uuid()` server-side generation (ids are minted app-side, as they already are for projects/devices) — accepted, because it matches the existing code and avoids a store-wide id-type migration.

1.1 `Account` — the tenant (top of the tree)

The account is the top of the hierarchy `account → projects → OTA updates`. It has no parent. Account-level identifiers are legitimately **global-unique** (accounts ARE the tenant boundary, so there is no enclosing scope).

Field	Type	Notes
`account_id`	`TEXT` **PK**	opaque, app-minted (mirrors `project_id`)
`name`	`TEXT NOT NULL`	display name; **UNIQUE (global)** — one tenant per name in the super-admin console

Field	Type	Notes
slug	TEXT NOT NULL	URL-/token-safe stable handle; **UNIQUE (global)**; the stable key an account switcher / CLI selects by (§11.4.111 resolve-by-stable-name)
status	TEXT NOT NULL DEFAULT 'active'	CHECK IN ('active','suspended','archived') — suspend gates all sign-in/scoped access without deleting data
created_at	TIMESTAMPTZ NOT NULL	
updated_at	TIMESTAMPTZ NOT NULL	

- **PK:** `account_id`. **Unique:** `name`, `slug` (both global).
- Go type mirrors `store.Project` (`store.go:289-295`) one level up.

1.2 `User` — persisted identity (closes the §3 gap)

Today there is **no persisted user** — identity is the bare token `Subject` string and an in-memory env-seeded map (10_* §3; `token.go:32`, `users.go:8-48`, `main.go:96-104`). This design persists the User the target migration already sketched (`001_initial_schema.up.sql:39-51`). A User is a **single global identity** that can belong to many accounts (the mandate: "users belong to one OR more accounts", 00_INDEX §1.3) — so user identifiers are global-unique and the *account relationship* lives in Membership (§1.3), not on the user row.

Field	Type	Notes
user_id	TEXT **PK**	opaque, app-minted
username	TEXT NOT NULL	login identity; **UNIQUE (global)**; the value the token `Subject` resolves to
email	TEXT NOT NULL	**UNIQUE (global)**
password_hash	TEXT NOT NULL	credential hash only, never cleartext (mirrors target `users.password_hash`, `001_initial_schema.up.sql:43`)
is_super_admin	BOOLEAN NOT NULL DEFAULT FALSE	the **global** super-admin flag (§6) — bypasses account scoping; set ONLY via config/

Field	Type	Notes
		env bootstrap, never a request (§11.4.10; 10_* §7)
`is_active`	`BOOLEAN NOT NULL DEFAULT TRUE`	disable without delete
`created_at` / `updated_at`	`TIMESTAMPTZ NOT NULL`	

- **PK:** `user_id`. **Unique:** `username`, `email`.
- **Resolves the CallerID ambiguity** (10_* §8 assumption 5): the token `Subject` (a username) resolves to `user_id`; memberships (§1.3) then attach roles per account. `ProjectAccess.CallerID` (`store.go:307-311`) should migrate to reference `user_id` (reconciliation item, §2).

1.3 `Membership` — user ↔ account (the many-to-many)

A user belongs to one OR more accounts (00_INDEX §1.3); an account has many users. This is the direct generalization of the existing `ProjectAccess` (caller↔project↔role, `store.go:306-311`) lifted one level to account↔user↔role. It is the table the WorkOS multi-tenant RBAC model calls the membership join (`user_roles` keyed `(user_id, tenant_id, role_id)`).

Field	Type	Notes
`user_id`	`TEXT NOT NULL`	**FK** → `users(user_id) ON DELETE CASCADE`
`account_id`	`TEXT NOT NULL`	**FK** → `accounts(account_id) ON DELETE CASCADE`
`role`	`TEXT NOT NULL DEFAULT 'viewer'`	per-account role (§1.4); `CHECK IN ('viewer','operator','admin')`
`is_owner`	`BOOLEAN NOT NULL DEFAULT FALSE`	the account's owner (the membership minted at bootstrap, §3.3); at most one owner per account is a policy `21_*` decides
`granted_at`	`TIMESTAMPTZ NOT NULL`	
`granted_by`		

Field	Type	Notes
	`TEXT NOT NULL DEFAULT ''`	granting user_id (super-admin or account admin)

- **PK:** composite `(user_id, account_id)` — one membership row per (user, account); a user in N accounts has N rows. **Role is per-account**, so the same user can be `admin` in account A and `viewer` in account B (WorkOS: "the same user may hold different roles in different contexts").
- Go type: `AccountMembership{AccountID, UserID string; Role AccountRole; IsOwner bool}`.

1.4 `Role` / `Permission` — the per-account authorization vocabulary

The mandate asks for "ultimate flexibility to the maximum" (00_INDEX §1.6), but the *shape* of the permission model (RBAC vs ABAC vs role+scope hybrid) is **OQ-2, explicitly owned by `21_authz_rbac_superadmin.md`** (00_INDEX §5). This keystone doc models only the minimal role skeleton the data layer needs and reserves the richer path so `21_*` can land it without a second data-model change.

Recommendation for THIS doc (defer full shape to `21_*`): ship the fixed per-account role enum on `Membership.role` now, mirroring the existing `ProjectRole` hierarchy `viewer < operator < admin` (`store.go:297-304`) so the existing rank-compare tooling (`handlers_project.go:78-92`) reuses unchanged. `AccountRole` = `viewer < operator < admin`, distinct from but structurally identical to `ProjectRole`.

Reserved "max-flexibility" path (roles/permissions tables), if `21_*` chooses it: a tenant-scoped RBAC set — `roles(role_id, account_id, name, UNIQUE(account_id,name))`, `permissions(permission_id, action, resource)`, `role_permissions(role_id, permission_id)`, and `Membership.role` becoming a `role_id` FK — the "hybrid/templates" model WorkOS recommends (default roles + per- tenant customization). **Tradeoff:** fixed-enum is simplest and reuses existing tooling but caps flexibility at three roles; the roles/permissions tables give the mandated "ultimate flexibility" at the cost of a role-resolution join on every authZ check. Presented for `21_*`; §4 emits the fixed-enum column now + the reserved tables as commented DDL so either path is a pure additive migration.

1.5 How Account sits ABOVE Project

```
Account (tenant)                          accounts
  └─ Project (add account_id FK)          projects.account_id →
accounts
       └─ OTA updates:                    (each carries account_id
[RLS key] + project_id)
            Artifact, Release,
            Deployment, Device,
            Group, Telemetry
```

The as-is gap (10_* §4): projects are **empty containers** — Device/ Artifact/ Release/Deployment/Telemetry/Group carry **no `ProjectID`** (`store.go:36-178`), so the `account → projects → OTA updates` link is entirely absent below the project row. §2 adds it.

---

2. Scoping the existing OTA model

Design rule (from §0 recommendation + AWS RLS best practice): every tenant-owned table gets a **denormalized `account_id` (NOT NULL, the RLS scoping key, and the leading column of its primary access index** — AWS: "every table with an RLS policy needs `tenant_id` as the leading column in its primary access indexes"), AND a `project_id` (the true mid-tier parent). `account_id` is denormalized onto each resource rather than reached only via `project_id → account_id` because RLS policies and the hot scoping filter must not pay a join on every row.

Tradeoff of the denormalization: `resource.account_id` must always equal `project.account_id`. Enforce with a **composite FK** `(project_id, account_id) REFERENCES projects(project_id, account_id)` (requires a `UNIQUE(project_id, account_id)` on projects) so the database — not application discipline — guarantees the two never drift. Alternative (a trigger) is weaker; the composite FK is "original work" applied to this schema and is the recommended enforcement.

2.1 Per-entity column plan

Entity (struct / table)	add `account_id`	add `project_id`	Nullability	Migration / backf implication
	yes (it is the mid-	n/a	`account_id` **NOT NULL**	backfill every existing project to

Entity (struct / table)	add `account_id`	add `project_id`	Nullability	Migration / backf implication
`Project` (`store.go:289`; `projects` `schema_postgres.sql:239`)	tier, not the top)		after backfill	the default accoun (§5)
`Device` (`store.go:36-*`; `devices :14`)	yes	yes	`account_id` **NOT NULL**; `project_id` **NULL-until-assigned** (a device may register to an account before project assignment)	backfill account; `project_id` may sta NULL
`Artifact` (`artifacts :33`)	yes	yes	both **NOT NULL** after backfill (artifacts are uploaded within a project)	backfill account + default project
`Release` (`releases :47`)	yes	yes	both **NOT NULL**	scopes the monotonicity key (§2.2 #3)
`Deployment` (`deployments :61`)	yes	yes	both **NOT NULL**	scopes the active-uniqueness key (§ #4)
`TelemetryRecord` (`telemetry_events :73`)	yes	inherit via device/ deployment	`account_id` **NOT NULL**; `project_id` **NULLABLE**	derive from the referenced device/ deployment at wri time
`Group` (`device_groups :96`)	yes	yes	`account_id` **NOT NULL**; `project_id` **NULLABLE** (a group may be	name becomes unique-per-accoun (§2.2 #6)

Entity (struct / table)	add `account_id`	add `project_id`	Nullability	Migration / backf implication
			account-wide)	
`AuditEntry` (`audit_logs :115`; `store.go:123-134`)	yes	yes	both **NULLABLE**	populate from the resolved account/ project context; als finally populate th always-empty `User` (10_* §6, `audit_wire.go:41-`

2.2 The 7 single-global-tenant assumptions (10_* §8) → account-scoped

Each global uniqueness/latest key becomes account- (and where relevant project-) scoped:

1. **Project name globally unique** (`schema_postgres.sql:241`; `memory.go:39` `prjByName`) → **`UNIQUE (account_id, name)`**. Two tenants may both have a project "production".
2. **Device `hardware_id` globally unique** (`schema_postgres.sql:30`; `store.go:338` `GetDeviceByHardwareID`) → **`UNIQUE (account_id, hardware_id)`**, and `GetDeviceByHardwareID` gains an `accountID` parameter (§3.2). *Tradeoff / flag:* if a single physical device could legitimately belong to two accounts this must instead be a global unique — but the mandate's tree makes a device account-owned, so per-account is recommended; surfaced for operator confirmation.
3. **Release monotonicity / "latest" keyed `(os_type, target_model)`** (`store.go:349` `LatestRelease`; `releaseMu` invariant `server.go:56-69`) → **`(account_id, project_id, os_type, target_model)`**. `LatestRelease` / `ReleaseByVersion` gain account+project scope (§3.2).
4. **Active-deployment uniqueness keyed `(os, target_model, group)`** (`store.go:359` `ActiveDeploymentForTarget`; `deployMu` `server.go:42-54`) → **`(account_id, project_id, os, target_model, group)`**.
5. **`ProjectAccess.CallerID` a bare subject string** (`store.go:307-311`) → resolves to `user_id` (§1.2); the same person in several accounts is now representable because membership (§1.3) is keyed per account. `project_access` rows are implicitly account-scoped via `projects.account_id`.
6. **Group name globally unique** (`schema_postgres.sql:102`; `memory.go` `grpByName`) → **`UNIQUE (account_id, name)`**.

7. **Token `Claims` carry no tenant dimension** (`token.go:31-36`) → the data model requires either an `account_id` claim in the minted token OR a per-request membership lookup. **The token-shape choice is `21_*`'s** (OQ-2). The *data-model requirement* this doc fixes: the Membership table MUST support a fast `ListUserAccounts(userID)` (for account selection after sign-in, 00_INDEX §1.5) and `GetAccountMembership(userID, accountID)` (per-request authZ) — both indexed in §4. **Recommendation to `21_*` (not decided here):** resolve membership per-request (supports account switching without re-minting) with an optional `account_id` claim stamped *after* the user selects an account; tradeoff is a per-request lookup vs a re-mint on switch.

---

3. `store.Repository` extension

The Repository is one flat interface (`store.go:315-429`) with two implementations — `MemoryRepository` (default, `main.go:80`) and `PostgresRepository` (`HELIX_DATABASE_URL` set, `main.go:46-78`). The existing Projects block (`store.go:317-332`) is the exact template. The SAME interface methods are added once and implemented by BOTH backends (10_* §5: "additive-friendly — yes").

3.1 New interface methods (mirror the Projects block)

```
// Accounts (mirror CreateProject/Get/List/Update/Delete,
        store.go:317-321).
CreateAccount(ctx context.Context, a Account) error
GetAccount(ctx context.Context, accountID string) (Account, error)
GetAccountBySlug(ctx context.Context, slug string) (Account,
        error) // stable-name resolve (§11.4.111)
ListAccounts(ctx context.Context) ([]Account,
        error)              // super-admin only (§6)
UpdateAccount(ctx context.Context, a Account) error
DeleteAccount(ctx context.Context, accountID string) error

// Users — the CRUD that does NOT exist today (10_* §3).
CreateUser(ctx context.Context, u User) error
GetUser(ctx context.Context, userID string) (User, error)
GetUserByUsername(ctx context.Context, username string) (User,
        error) // token Subject → user
GetUserByEmail(ctx context.Context, email string) (User, error)
ListUsers(ctx context.Context) ([]User,
        error)                        // super-admin only
UpdateUser(ctx context.Context, u User) error
```

```
DeleteUser(ctx context.Context, userID string) error

// Memberships (mirror the ProjectAccess block, store.go:322-332).
SetAccountMembership(ctx context.Context, m AccountMembership)
        error   // grant/update role
GetAccountMembership(ctx context.Context, userID, accountID
        string) (AccountMembership, error)
ListAccountMembers(ctx context.Context, accountID string)
        ([]AccountMembership, error)
ListUserAccounts(ctx context.Context, userID string)
        ([]AccountMembership, error) // account selection after
        sign-in
RemoveAccountMembership(ctx context.Context, userID, accountID
        string) error
```

3.2 Scoping the existing read/write methods

Every existing OTA accessor must become account-scoped. Two mechanisms, with a recommended hybrid:

- **Option A — explicit `accountID` parameter on every scoped method.** e.g. `GetDevice(ctx, accountID, deviceID)`, `GetDeviceByHardwareID(ctx, accountID, hardwareID)`, `LatestRelease(ctx, accountID, projectID, os, targetModel)`, `ActiveDeploymentForTarget(ctx, accountID, projectID, os, targetModel, group)`. *Pro:* the compiler forces every call site to supply scope — a forgotten scope is a build error, not a runtime leak. *Con:* churns every signature + call site.
- **Option B — carry scope in `context.Context`** (a `ScopeContext{AccountID, ProjectID}` read by each method). *Pro:* minimal signature churn; mirrors the RLS session-variable pattern (set once per request). *Con:* scope becomes implicit — a call in a context missing the scope silently reads unscoped (the exact leak RLS must then catch).
- **Existing `*Filter` structs already exist** for the list methods (`DeviceFilter` `store.go:280-285`, `ReleaseFilter`, `AuditFilter`) — add `AccountID` + `ProjectID` fields there for the list paths with zero signature change.

Recommendation (hybrid): Option A (explicit `accountID`, and `projectID` where the resource is project-owned) for the single-entity get/create/update methods + extend the `*Filter` structs for the list methods, with **pgx RLS as the independent second layer** so even a missed scope cannot leak (defense in depth). Explicit params are chosen over ctx-implicit specifically because the whole point of this feature is tenant isolation, and a compile-time guarantee beats a

runtime one. *Tradeoff:* larger diff (every OTA handler + call site touched) — acceptable, and `22_api_surface.md` already inventories those call sites.

3.3 The missing-transaction problem (atomic "create account + first super-admin")

The seam has **no transaction primitive** — the code explicitly does a best-effort, non-atomic create-then-grant for projects (10_* §5; `handlers_project.go:137-154`). For accounts this is worse: a crash between "account created" and "owner membership granted" leaves an **ownerless orphan tenant** — nobody can administer it and no membership gates it (an isolation + usability defect). Three seam options:

- **Option A — a purpose-built composite method** `CreateAccountWithOwner(ctx, a Account, ownerUserID string, role AccountRole) error`, implemented atomically by each backend (pgx: `BEGIN … COMMIT`; memory: a single mutex-guarded critical section that inserts both the account and the owner membership, rolling back the in-memory map insert on any error). *Pro:* smallest seam change; matches the existing pattern of purpose-built methods; solves the one flow that actually needs atomicity now. *Con:* a bespoke method per atomic flow (doesn't generalize).
- **Option B — a generic transaction primitive** `WithTx(ctx, func(RepositoryTx) error) error` on the interface (pgx: a real transaction; memory: a global-lock stand-in). *Pro:* general — any future multi-write flow (create project + grant, deploy + audit) becomes atomic. *Con:* large seam change; the memory impl's "transaction" is only a coarse lock (no true rollback of partial map mutations without extra bookkeeping); bigger blast radius.
- **Option C — keep best-effort compensation** (create account → grant → on grant failure, delete account). *Pro:* zero seam change. *Con:* non-atomic across a crash (the orphan-tenant window remains) — rejected for the security-sensitive bootstrap.

Recommendation: Option A now (`CreateAccountWithOwner`, the one flow that must be atomic for correctness), and **adopt Option B (`WithTx`) later** when a second multi-write atomic flow appears — deferred to the implementation phase, not required for MVP. *Tradeoff:* Option A doesn't generalize, but it is the minimal change that closes the orphan-tenant hole; Option B is the right long-term seam but is over-scoped for a single bootstrap flow today.

---

4. SQL schema sketch (migration-style DDL)

A store-schema-style migration (opaque `TEXT` ids, mirroring `schema_postgres.sql`) — call it **`002_accounts_multitenancy.up.sql`**, additive to the shipped store. The canonical `UUID` variant would land the same tables in the `001`-style schema (`001_initial_schema.up.sql`); the mapping is `TEXT ↔ UUID`, `users`/`api_keys` here match `001_initial_schema.up.sql:39-71`. RLS statements are the recommended second isolation layer (§0). This is a **sketch for the design; not applied**.

```
-- 002_accounts_multitenancy (UP) — additive to
        schema_postgres.sql. TEXT ids to
-- match the shipped store; UUID in the canonical 001-style
        schema. NOT executed.
SET search_path = helix_ota, public;


-- 1) New top-level tenant.
CREATE TABLE IF NOT EXISTS helix_ota.accounts (
    account_id  TEXT PRIMARY KEY,
    name        TEXT        NOT NULL UNIQUE,
    slug        TEXT        NOT NULL UNIQUE,
    status      TEXT        NOT NULL DEFAULT 'active'
                 CHECK (status IN
        ('active','suspended','archived')),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 2) Persisted User (closes the 10_* §3 gap; mirrors
        001_initial_schema users).
CREATE TABLE IF NOT EXISTS helix_ota.users (
    user_id        TEXT PRIMARY KEY,
    username       TEXT        NOT NULL UNIQUE,
    email          TEXT        NOT NULL UNIQUE,
    password_hash  TEXT        NOT NULL,
    is_super_admin BOOLEAN     NOT NULL DEFAULT FALSE,   -- §6
        global bypass flag
    is_active      BOOLEAN     NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 2b) api_keys (from 001_initial_schema.up.sql:57-71; non-
        interactive CI/CLI creds).
CREATE TABLE IF NOT EXISTS helix_ota.api_keys (
```

```
    api_key_id  TEXT PRIMARY KEY,
    user_id     TEXT        NOT NULL REFERENCES
        helix_ota.users(user_id) ON DELETE CASCADE,
    account_id  TEXT        NOT NULL REFERENCES
        helix_ota.accounts(account_id) ON DELETE CASCADE,
    key_hash    TEXT        NOT NULL UNIQUE,           -- hash
        only; cleartext shown once (§11.4.10)
    name        TEXT        NOT NULL,
    permissions JSONB       NOT NULL DEFAULT '{}'::jsonb,
    expires_at  TIMESTAMPTZ,
    revoked_at  TIMESTAMPTZ,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_api_keys_user    ON
        helix_ota.api_keys (user_id);
CREATE INDEX IF NOT EXISTS idx_api_keys_account ON
        helix_ota.api_keys (account_id);

-- 3) Membership (user ↔ account, M:N) — mirrors project_access
        one level up.
CREATE TABLE IF NOT EXISTS helix_ota.account_members (
    user_id    TEXT        NOT NULL REFERENCES
        helix_ota.users(user_id)       ON DELETE CASCADE,
    account_id TEXT        NOT NULL REFERENCES
        helix_ota.accounts(account_id) ON DELETE CASCADE,
    role       TEXT        NOT NULL DEFAULT 'viewer'
                 CHECK (role IN ('viewer','operator','admin')),
    is_owner   BOOLEAN     NOT NULL DEFAULT FALSE,
    granted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    granted_by TEXT        NOT NULL DEFAULT '',
    PRIMARY KEY (user_id, account_id)
);
CREATE INDEX IF NOT EXISTS idx_account_members_account ON
        helix_ota.account_members (account_id);
CREATE INDEX IF NOT EXISTS idx_account_members_user    ON
        helix_ota.account_members (user_id);

-- 3b) RESERVED "max-flexibility" RBAC (only if 21_* chooses it —
        additive, not now):
-- CREATE TABLE helix_ota.roles (role_id TEXT PK, account_id TEXT
        NOT NULL
--   REFERENCES accounts(account_id) ON DELETE CASCADE, name TEXT
        NOT NULL,
--   UNIQUE (account_id, name));
-- CREATE TABLE helix_ota.permissions (permission_id TEXT PK,
        action TEXT NOT NULL,
--   resource TEXT NOT NULL, UNIQUE (action, resource));
-- CREATE TABLE helix_ota.role_permissions (role_id TEXT,
        permission_id TEXT,
--   PRIMARY KEY (role_id, permission_id));
```

```
-- and account_members.role becomes a role_id FK.

-- 4) Scope the existing OTA tables. account_id nullable now → backfill → NOT NULL
--    in 003 (see §5 Option A). project_id added to the resources per §2.1.
ALTER TABLE helix_ota.projects          ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.devices           ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.devices           ADD COLUMN IF NOT EXISTS project_id TEXT;   -- NULL-until-assigned
ALTER TABLE helix_ota.artifacts         ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.artifacts         ADD COLUMN IF NOT EXISTS project_id TEXT;
ALTER TABLE helix_ota.releases          ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.releases          ADD COLUMN IF NOT EXISTS project_id TEXT;
ALTER TABLE helix_ota.deployments       ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.deployments       ADD COLUMN IF NOT EXISTS project_id TEXT;
ALTER TABLE helix_ota.device_groups     ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.device_groups     ADD COLUMN IF NOT EXISTS project_id TEXT;   -- nullable
ALTER TABLE helix_ota.telemetry_events  ADD COLUMN IF NOT EXISTS account_id TEXT;
ALTER TABLE helix_ota.telemetry_events  ADD COLUMN IF NOT EXISTS project_id TEXT;   -- nullable
ALTER TABLE helix_ota.audit_logs        ADD COLUMN IF NOT EXISTS account_id TEXT;   -- nullable
ALTER TABLE helix_ota.audit_logs        ADD COLUMN IF NOT EXISTS project_id TEXT;   -- nullable

-- 5) Composite-FK enforcement so resource.account_id == project.account_id (§2).
--    Requires a UNIQUE (project_id, account_id) on projects to be the FK target.
--    (Deferred to 003 after backfill sets account_id, since project_id is the PK
--     and account_id starts nullable.)
-- ALTER TABLE helix_ota.projects   ADD CONSTRAINT projects_pid_aid_uniq UNIQUE (project_id, account_id);
-- ALTER TABLE helix_ota.artifacts  ADD CONSTRAINT artifacts_proj_fk
--     FOREIGN KEY (project_id, account_id) REFERENCES helix_ota.projects (project_id, account_id);
-- (same composite FK for releases, deployments; devices/groups/telemetry where project_id NOT NULL)
```

```
-- 6) Uniqueness re-scoping (§2.2). Old global uniques are
       dropped, per-account added
--    in 003 after backfill (a per-account unique cannot be added
       while account_id is
--    still NULL for legacy rows).
-- ALTER TABLE helix_ota.projects      DROP CONSTRAINT ... ; ADD
       UNIQUE (account_id, name);
-- ALTER TABLE helix_ota.devices       DROP CONSTRAINT
       devices_hardware_id_uniq;
--   ADD CONSTRAINT devices_acct_hw_uniq UNIQUE (account_id,
       hardware_id);
-- ALTER TABLE helix_ota.device_groups DROP CONSTRAINT
       device_groups_name_uniq;
--   ADD CONSTRAINT groups_acct_name_uniq UNIQUE (account_id,
       name);

-- 7) Indexes: account_id as the LEADING column of each hot
       scoping index (AWS RLS
--    indexing rule). Added in 003 once account_id is populated +
       NOT NULL.
-- CREATE INDEX idx_devices_account     ON helix_ota.devices
       (account_id);
-- CREATE INDEX idx_artifacts_account   ON helix_ota.artifacts
       (account_id);
-- CREATE INDEX idx_releases_account    ON helix_ota.releases
       (account_id, project_id, os_type, target_model);
-- CREATE INDEX idx_deployments_account ON helix_ota.deployments
       (account_id, project_id, status);
-- CREATE INDEX idx_telemetry_account   ON
       helix_ota.telemetry_events (account_id);
-- CREATE INDEX idx_audit_account       ON helix_ota.audit_logs
       (account_id);

-- 8) RLS (recommended §0 second layer; enable in 003 after NOT
       NULL). Per-table:
-- ALTER TABLE helix_ota.devices ENABLE ROW LEVEL SECURITY;
-- CREATE POLICY devices_tenant_isolation ON helix_ota.devices
--     USING       (account_id =
       current_setting('app.current_account', true))
--     WITH CHECK  (account_id =
       current_setting('app.current_account', true));
--    The app connects as a NON-owner, NON-BYPASSRLS role and runs
--    `SET app.current_account = '<account_id>'` per request (AWS
       pattern). A
--    super-admin request either sets a super-admin GUC the policy
       also honors, or
--    is served by a policy-based bypass (§6) — NOT a BYPASSRLS
       role.
```

> **Reference to the canonical target schema.** `users` + `api_keys` above mirror `001_initial_schema.up.sql:39-71` (which the shipped store never carried, 10_* §3). The `account_id`/`project_id`

> scoping columns are the additive complement to that migration's device/artifact/release/deployment tables (`001_initial_schema.up.sql:80-360`). When the project adopts the `001`-style normalized schema, this becomes `002` there too, with `UUID` ids and FK-to-`users` on `uploaded_by`/`published_by`/`created_by` (`001_initial_schema.up.sql:155,219,265`) finally resolvable.

---

5. OQ-4 — migrating existing data under accounts

OQ-4 (00_INDEX §5): migrate existing projects/rollouts/artifacts under a **default account**, or keep the schema **additive-nullable until backfill**? Two options:

- **Option A — default-account backfill (big-bang, then NOT NULL).** `002` adds `account_id` nullable → a one-shot backfill assigns every existing project, device, artifact, release, deployment, group, telemetry, audit row to a single seeded **`__default__` system account** (and a `__default__` project where a project_id is required) → `003` sets `account_id NOT NULL`, adds the per-account uniques, the composite FKs, the leading-column indexes, and enables RLS. *Pro:* one clean invariant (`account_id` is NEVER NULL for a tenant row after `003`); RLS is correct immediately; no "NULL means legacy-global" special case litters the code. *Con:* requires a data-migration step and a brief coordinated `003` cutover.
- **Option B — additive-nullable-until-backfill (gradual).** Add `account_id` nullable and ship code that tolerates NULL as "unassigned/legacy-global", backfill lazily, flip NOT NULL "eventually." *Pro:* purely additive, zero-downtime, no cutover. *Con:* while `account_id` is NULL, those rows are **outside** per- account scoping — an isolation hole for the whole window; every read path needs a NULL branch; the NOT-NULL flip still needs the same backfill Option A does, just later and with more accumulated NULL rows.

Recommendation: Option A (default-account backfill). It removes the nullable- isolation-hole window entirely and yields a clean NOT-NULL + RLS invariant on day one. The deciding factor is **data volume**, and the as-is state makes the backfill nearly free: the store is an **in-memory MVP by default** (`main.go:80`) with Postgres wired only when `HELIX_DATABASE_URL` is set and barely populated — so the big-bang cost Option A normally carries is negligible here.

Operator decision point (flagged, not decided): if, contrary to the as-is state, a target deployment already holds meaningful production OTA data AND a zero-downtime cutover is mandatory, choose Option B and accept the interim NULL- scoping window with an explicit "NULL account_id is visible only to super-admin" guard. The recommendation flips on that single fact — surfaced per §11.4.6, not assumed.

---

6. Super-admin at the data layer

The mandate: the super-user "sees and controls everything" and administers all accounts/users/permissions (00_INDEX §1.4, §1.6). Model it as a **global `users.is_super_admin` boolean** (§1.2) — a property of the *identity*, NOT a per-account membership role (a super-admin need not be a member of any account).

How the flag bypasses account scoping (two layers):

1. **Application layer.** The scope resolver returns "all accounts" when the authenticated user is super-admin — the *exact precedent already in the code*: `requireProjectAccess` lets a global `admin` bypass the per-project ACL and implicitly access every project (`handlers_project.go:43-60`), and the list-all path returns every project for a global admin (`handlers_project.go:162-199`). The account layer generalizes this: super-admin ⇒ skip the `account_id` filter / `requireAccountAccess`.
2. **Database layer (RLS).** Prefer a **policy-based bypass over a `BYPASSRLS` role.** A normal request connects as a non-owner, non-`BYPASSRLS` role and sets `app.current_account`; a super-admin request additionally sets a super-admin GUC (e.g. `SET app.is_super_admin = 'on'`) that each policy's `USING` clause also accepts: `USING (account_id = current_setting('app.current_account', true) OR current_setting('app.is_super_admin', true) = 'on')`. AWS explicitly recommends "policy-based admin access over privilege-based bypass" because it "keeps admin access **auditable and revocable**" — a `BYPASSRLS` role is invisible to the policy and cannot be scoped or logged per query. (PostgreSQL: superusers and `BYPASSRLS` roles *always* bypass RLS — which is exactly why the app must NOT connect as one for super-admin requests.)

Security implications (data layer; full model in `26_security_threat_model.md`):

- **Maximum blast radius.** One compromised super-admin credential = total cross- tenant breach. Therefore `is_super_admin` is settable **only via config/env bootstrap, never a request** — the same trust-boundary rule the codebase already enforces for the token secret (`config.go:180-184`), the admin bootstrap (`main.go:96-104`), the TLS-proxy trust flag (`config.go:88-110`), and the artifact-signature key `resolvePublicKey` (`handlers_artifact.go:274-283`, which takes the verify key ONLY from server config, never the request). The first super-admin is seeded from a `HELIX_*` env field (or the existing `HELIX_ADMIN_PASSWORD` path extended with `is_super_admin=true`), following 10_* §7 exactly.
- **Auditability.** Every super-admin action MUST record the **affected tenant's `account_id`** in the audit row (§2.1; the audit table today carries no account_id — 10_* §6). The policy-based bypass (not a BYPASSRLS role) is what keeps those actions attributable per query.
- **Testable, revocable (anti-bluff).** `26_*` must carry a paired mutation proving
  1. a non-super-admin session with `app.current_account = A` **cannot** read account B's rows even with a forged `WHERE`, and (b) clearing `is_super_admin` (or the GUC) **immediately** re-scopes the session — a super-admin bypass that survives revocation is a §11.4 isolation-layer bluff.

Sources verified 2026-07-10

- PostgreSQL Row-Level Security for multi-tenant isolation — runtime tenant context via `SET`/`current_setting('app.current_tenant')`, the app-role-must-not-be-owner- or-`BYPASSRLS` caveat, and the "prefer policy-based admin access over privilege- based bypass … auditable and revocable" guidance: https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/
- PostgreSQL official docs — canonical RLS primitives (`ALTER TABLE … ENABLE ROW LEVEL SECURITY`, `CREATE POLICY … USING … WITH CHECK`, `FORCE ROW LEVEL SECURITY`, superuser/`BYPASSRLS` bypass): https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- WorkOS — multi-tenant RBAC data model: membership join keyed `(user_id, tenant_id, role_id)`, global vs tenant-scoped vs hybrid/template roles, `UNIQUE(tenant_id, name)` for tenant-scoped role

namespaces: https://workos.com/blog/how-to-design-multi-tenant-rbac-saas
- Microsoft Azure Architecture Center — tenancy models (shared vs schema-per vs database-per) and the isolation/ops-cost tradeoff taxonomy: https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models and Azure SQL multitenant SaaS patterns: https://learn.microsoft.com/en-us/azure/azure-sql/database/saas-tenancy-app-design-patterns
- Supplementary comparison of the three tenancy patterns (shared-schema+RLS as the default recommendation): dasroot 2026, https://dasroot.net/posts/2026/01/multi-tenancy-database-patterns-schema-database-row-level/

The composite-FK enforcement of `resource.account_id == project.account_id` (§2) and the `CreateAccountWithOwner` atomic-bootstrap seam (§3.3) are **original work** applied to this schema — no external source prescribes them; they follow from the shipped store's shape.

Honest boundary (§11.4.6)

- **This is a design proposal, not implemented.** No entity, column, interface method, migration, or RLS policy described here exists in the codebase yet; every "as-is" fact carries a `file:line` (via 00_/10_*), every "to-be" element is a proposal. Nothing here is a §11.4 completion claim.
- **Every open choice is presented with a recommendation + tradeoffs for operator decision, never silently decided:** the tenancy model (§0), the id type TEXT-vs-UUID divergence (§1), the OTA-resource scoping shape (§2), the Repository scoping mechanism and the atomic-bootstrap seam (§3), the OQ-4 migration path (§5), the super-admin RLS-bypass mechanism (§6). The permission-model *shape* (OQ-2) is deliberately deferred to `21_authz_rbac_superadmin.md`; this doc fixes only the minimal role skeleton + the data-layer requirements `21_*` must satisfy.
- **Scope of the audit behind the as-is facts:** the `file:line` references were established by 10_* against non-test source only; a test path may exercise a detail not seen. The pgx read paths were read as SQL strings, not executed against a live DB (10_* "Honest gaps"). The rollout subsystem's tenancy (`internal/rollout/`) is marked `UNCONFIRMED:` in 10_* and is not designed here — `21_*`/`30_*` must audit `rollout/store.go` before scoping it.

- **What the recommendation does NOT prove:** shared-schema+RLS is the correct *default*, not a guarantee of isolation — isolation is only as good as the policies
  - the scope-resolution code, which is precisely why `26_security_threat_model.md` owns the paired-mutation proof that cross-account reads are impossible.